0141. Linked List Cycle

: Easy | Category: Linked List | LeetCode: https://leetcode.com/problems/linked-l

Problem

- Return true if the linked list contains a cycle. Follow-up: O(1) memory.

Intuition

- Floyd's tortoise and hare: slow moves 1 step, fast moves 2.
- In a cycle, fast gains one node per step on slow, so they must meet. Without a cycle, fast reaches null.

Brute Force

- Store visited nodes in a HashSet: O(n) space.

Optimal Approach

- Slow = fast = head.
- While fast and fast.next: slow = slow.next; fast = fast.next.next; if slow == fast return true.
- Return false.

Worked Example

```
3 -> 2 -> 0 -> -4 -> (back to 2).

step1 slow=2  fast=0
step2 slow=0  fast=2
step3 slow=-4 fast=-4  meet -> true
```

Java Solution

```java
public class LinkedListCycle {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- Compare node references, not values.